

Python 42 — Recap Complet

Modules 00, 01, 02 et 03 — Tout en un

Theorie, syntaxe, exemples et bonnes pratiques

Preparation : Entretien technique Odoo Belgium

Table des matieres

MODULE 00 — Les bases

1. print() et input()
2. Les variables et types de base
3. Les conversions de types
4. Les f-strings et la concatenation
5. Les conditions — if / elif / else
6. Les boucles — for, range()
7. Les fonctions — def, return
8. Les parametres avec valeur par default
9. Les methodes de string
10. La recursion
11. Le shebang et if `__name__ == '__main__'`

MODULE 01 — Programmation Orientee Objet

12. Les classes — class, `__init__`, self
13. L'heritage — super()
14. L'encapsulation — convention `_attribut`
15. Les getters et setters + verbose
16. Le polymorphisme — override
17. Les methodes statiques — `@staticmethod`
18. Les methodes de classe — `@classmethod`
19. Les classes imbriquees
20. Les fonctions libres

MODULE 02 — Gestion des exceptions

21. try / except — attraper une erreur
22. raise — lancer une erreur
23. Plusieurs types d'erreurs
24. finally — toujours nettoyer
25. Exceptions personnalisees

MODULE 03 — Structures de donnees

26. sys.argv — arguments ligne de commande

27. Les listes

28. Les tuples

29. Les sets — ensembles

30. Les dictionnaires

31. Erreurs classiques à éviter

32. Bonnes pratiques PEP 8

MODULE 00 — Les bases

1. print() et input()

```
# print() : afficher du texte
print("Hello World!")
print("Valeur :", 42)           # plusieurs arguments separes par un espace
print("sans retour", end="")   # end="" supprime le saut de ligne
print(" a la suite")           # continue sur la meme ligne

# input() : demander une saisie utilisateur
# TOUJOURS renvoie une STRING, peu importe ce que l'utilisateur tape
nom = input("Entrez votre nom : ")
age = input("Votre age : ")     # age est une STRING ici !
age_int = int(age)              # il faut convertir pour faire des calculs
```

2. Les variables et types de base

```
int     age = 30                # entier
float   taille = 25.0           # nombre decimal
str     nom = "Rose"            # chaine de caracteres
bool    est_fleuri = False      # booleen : True ou False

# Verifier le type
type(age)      # <class 'int'>
type(taille)   # <class 'float'>
type(nom)      # <class 'str'>
```

3. Les conversions de types

```
int("42")      # "42" -> 42
float("3.14")   # "3.14" -> 3.14
str(42)         # 42 -> "42"

# Piege classique :
x = input("Nombre : ")      # x = "5" (string)
y = int(x) * 2              # y = 10 (int) – conversion obligatoire
# Sans conversion : TypeError ou comportement inattendu
```

4. Les f-strings et la concatenation

```
nom = "Rose"
hauteur = 25.0

# f-string (recommande) : prefixer la string avec f
print(f"Plante : {nom}")
print(f"{nom}: {hauteur}cm")      # Rose: 25.0cm
print(f"Double : {hauteur * 2}cm") # calcul dans la f-string
print(f"Arrondi : {round(hauteur, 1)}cm")

# Concatenation avec + (attention aux types !)
print("Plante : " + nom)          # OK : str + str
```

```
# print("Hauteur : " + hauteur)      # ERREUR : str + float !
print("Hauteur : " + str(hauteur))    # OK apres conversion

# print() avec virgule (espace automatique)
print("Plot area:", 15)              # Plot area: 15
```

5. Les conditions — if / elif / else

```
age = int(input("Age : "))

if age < 0 :
    print("Age invalide")
elif age < 18 :
    print("Mineur")
elif age < 65 :
    print("Adulte")
else :
    print("Senior")

# Operateurs de comparaison :
# ==  !=  >  <  >=  <=

# Operateurs logiques :
# and : les deux conditions doivent etre vraies
# or  : au moins une condition doit etre vraie
# not : inverse la condition

if hauteur > 50 and age > 30 :
    print("Grande plante mature")
```

6. Les boucles — for, range()

```
# range() : generer une sequence de nombres
range(5)          # 0, 1, 2, 3, 4
range(1, 6)       # 1, 2, 3, 4, 5
range(1, 10, 2)   # 1, 3, 5, 7, 9 (pas de 2)

# Boucle for classique
for i in range(1, 8) :
    print(f"Jour {i}")

# Boucle sur une liste
fruits = ["pomme", "banane", "cerise"]
for fruit in fruits :
    print(fruit)

# while : boucle conditionnelle
compteur = 0
while compteur < 5 :
    print(compteur)
    compteur += 1
```

```

# while True : boucle infinie controlee
while True :
    valeur = input("Entrez un nombre : ")
    try :
        nombre = int(valeur)
        break    # break : sort de la boucle immediatement
    except ValueError :
        print("Invalide, reessaie !")

# continue : passe directement a l'iteration suivante
for i in range(10) :
    if i % 2 == 0 :
        continue    # ignore les nombres pairs
    print(i)        # affiche seulement les impairs

```

7. Les fonctions — def, return

```

# Definition
def saluer(nom) :
    print(f"Bonjour {nom} !")

# Appel
saluer("Alice")

# Avec return : renvoie une valeur utilisable
def calculer_aire(longueur, largeur) :
    return longueur * largeur

resultat = calculer_aire(5, 3) # resultat = 15

# print() vs return :
# print() : affiche directement, ne renvoie rien d'utilisable
# return : renvoie la valeur pour l'utiliser ailleurs

# Type hints (bonne pratique) :
def calculer_aire(longueur: int, largeur: int) -> int :
    return longueur * largeur

```

8. Les parametres avec valeur par default

```

def saluer(nom: str, langue: str = "français") -> None :
    if langue == "français" :
        print(f"Bonjour {nom}")
    elif langue == "anglais" :
        print(f"Hello {nom}")

saluer("Marie")          # utilise "français" par default
saluer("Marie", "anglais") # remplace la valeur par default

# REGLE : les parametres avec valeur par default
# doivent toujours etre APRES ceux sans valeur par default

```

```
# def f(a, b=1) :    OK
# def f(a=1, b) :    ERREUR de syntaxe !
```

9. Les methodes de string

```
texte = "bonjour monde"

texte.capitalize()      # "Bonjour monde" (1ere lettre majuscule)
texte.upper()           # "BONJOUR MONDE"
texte.lower()           # "bonjour monde"
texte.title()           # "Bonjour Monde" (1ere lettre de chaque mot)
texte.strip()           # supprime espaces debut/fin
texte.replace("o", "0") # "b0njour m0nde"
texte.split(" ")        # ["bonjour", "monde"]
texte.split(",")        # ["bonjour monde"] (pas de virgule)
"1,2,3".split(",")     # ["1", "2", "3"]
len(texte)              # 13 (longueur)

# Tres utile pour parser des arguments :
argument = "sword:5"
parties = argument.split(":") # ["sword", "5"]
nom = parties[0]              # "sword"
quantite = int(parties[1])    # 5
```

10. La recursion

```
# Une fonction recursive s'appelle elle-meme
# Elle a TOUJOURS un cas de base (condition d'arret)

def compte(actuel: int, total: int) -> None :
    if actuel > total :    # cas de base : on s'arrete
        return
    print(f"Day {actuel}")
    compte(actuel + 1, total) # appel recursif avec valeur modifiee

compte(1, 5) # Day 1, Day 2, Day 3, Day 4, Day 5

# REGLES IMPORTANTES :
# 1. Toujours un cas de base sinon -> RecursionError (boucle infinie)
# 2. Chaque appel doit progresser VERS le cas de base
# 3. print() AVANT l'appel recursif = ordre 1,2,3...
#    print() APRES l'appel recursif = ordre 3,2,1... (inverse !)

# Comparaison iteratif vs recursif :
# Iteratif : utilise une boucle for/while
# Recursif : la fonction s'appelle elle-meme (pas de boucle)
```

11. Le shebang et if __name__ == '__main__'

```
#!/usr/bin/env python3
# ^ SHEBANG : toujours en PREMIERE ligne absolue du fichier
# Permet de lancer le script directement : ./mon_script.py
```

```
# Sans shebang : obligatoire de faire python3 mon_script.py
# Rendre executable : chmod +x mon_script.py

# if __name__ == "__main__" :
# Quand on execute directement : __name__ vaut "__main__"
# Quand on importe le fichier : __name__ vaut le nom du fichier

def ma_fonction() :
    print("Je suis une fonction")

if __name__ == "__main__" :
    # Ce bloc NE s'execute PAS si le fichier est importe
    # Il s'execute SEULEMENT si on lance directement
    ma_fonction()
```


MODULE 01 — Programmation Orientee Objet

12. Les classes — class, __init__, self

Une classe est un modele (plan de construction) qui decrit les donnees (attributs) et comportements (methodes) d'un type d'objet. Une instance est un objet concret cree a partir de ce modele.

```
class Plant :
    # __init__ : constructeur, appele automatiquement a la creation
    # self : represente l'instance en cours de creation
    def __init__(self, name: str, height: float, age: int) -> None :
        self.name = name          # attribut public
        self.height = height      # attribut public
        self.age = age            # attribut public

    # methode normale
    def show(self) -> None :
        print(f"{self.name}: {self.height}cm, {self.age} days old")

    def grow(self) -> None :
        self.height += 0.8        # modifie l'attribut de l'instance

# Instanciation (creation d'un objet)
rose = Plant("Rose", 25.0, 30)   # appelle __init__ automatiquement
oak = Plant("Oak", 200.0, 365)

# Appel de methodes
rose.show()                      # Rose: 25.0cm, 30 days old
oak.show()                      # Oak: 200.0cm, 365 days old

# Chaque instance est INDEPENDANTE :
rose.grow()                     # seulement rose est affectee, oak ne change pas
print(rose.height)              # 25.8
print(oak.height)               # 200.0 (inchange)
```

13. L'heritage — super()

L'heritage permet a une classe enfant de reutiliser tout le code d'une classe parente sans le reecrire. La classe enfant peut ajouter ses propres attributs/methodes ou redefinir (override) ceux du parent.

```
class Flower(Plant) :          # Flower herite de Plant
    def __init__(self, name: str, height: float, age: int, color: str) -> None :
        super().__init__(name, height, age)    # appelle __init__ du PARENT
        self.color = color                    # attribut SPECIFIQUE a Flower
        self.bloomed = False                 # toujours False au depart

    def bloom(self) -> None :
        self.bloomed = True

    def show(self) -> None :                # OVERRIDE de show() du parent
        super().show()                     # reutilise l'affichage du parent
```

```

        print(f"Color: {self.color}")
    if self.bloomed :
        print(f"{self.name} is blooming beautifully!")
    else :
        print(f"{self.name} has not bloomed yet")

# Chaine d'heritage : Seed herite de Flower qui herite de Plant
class Seed(Flower) :
    def __init__(self, name: str, height: float, age: int, color: str) -> None :
        super().__init__(name, height, age, color)
        self.seed_count = 0

    def bloom(self) -> None :
        super().bloom()      # passe bloomed a True
        self.seed_count = 10

    def show(self) -> None :
        super().show()       # affiche infos Plant + Flower
        print(f"Seed count: {self.seed_count}")

```

14. L'encapsulation — convention `_attribut`

```

# _ (un underscore) = convention "protected"
# Signale : "ne pas modifier directement depuis l'exterieur"
# Python ne l'interdit pas, mais c'est une convention a respecter

class Plant :
    def __init__(self, name: str, height: float) -> None :
        self.name = name          # public : accessible directement
        self._height = height     # protected : passer par les methodes

# NE PAS faire (violation de convention) :
rose._height = -50    # techniquement possible mais mauvaise pratique

# FAIRE : utiliser les methodes prevues (getters/setters)
rose.set_height(50)   # valide et securise

```

15. Les getters et setters + verbose

```

class Plant :
    def __init__(self, name: str, height: float, age: int) -> None :
        self.name = name
        self.set_height(height, verbose=False) # appel du setter des la creation
        self.set_age(age, verbose=False)

    # GETTER : lit et renvoie l'attribut protege
    def get_height(self) -> float :
        return self._height

    # SETTER : valide avant de modifier
    def set_height(self, value: float, verbose: bool = True) -> None :

```

```

    if value < 0 :
        print(f"{self.name}: Error, height can't be negative")
        print("Height update rejected")
    else :
        self._height = value
        if verbose :    # affiche le message seulement si verbose=True
            print(f"Height updated: {value}cm")

# verbose=True par default : affiche le message (appels normaux)
# verbose=False : silencieux (utilise dans __init__ pour eviter
# les messages parasites a la creation)

rose = Plant("Rose", 25.0, 30)    # silencieux (verbose=False)
rose.set_height(30)              # affiche "Height updated: 30cm"
rose.set_height(-5)              # affiche le message d'erreur

```

16. Le polymorphisme — override de methodes

```

# Des objets de classes differentes repondent au meme appel
# mais chacun avec son propre comportement

class Vegetable(Plant) :
    def show(self) -> None :    # override different de Flower.show()
        super().show()
        print(f"Harvest season: {self.harvest_season}")
        print(f"Nutritional value: {self.nutritional_value}")

# Le meme appel .show() produit des resultats differents :
rose = Flower("Rose", 15.0, 10, "red")
tomato = Vegetable("Tomato", 5.0, 10, "April")

rose.show()    # affiche infos Plant + couleur + statut floraison
tomato.show()  # affiche infos Plant + saison + valeur nutritive

```

17. Les methodes statiques — @staticmethod

```

# Appartient a la classe mais n'a pas acces a self
# N'a pas besoin d'une instance pour etre appelee
# Utile pour des fonctions utilitaires liees au domaine de la classe

class Plant :
    @staticmethod
    def is_older_than_a_year(age: int) -> bool :
        # Pas de "self" : pas d'accès a une instance
        if age <= 365 :
            return False
        return True

# Appel directement sur la CLASSE (pas besoin d'instance)
Plant.is_older_than_a_year(400)    # True
Plant.is_older_than_a_year(100)    # False

```

```
# Fonctionne aussi sur une instance (mais moins idiomatique)
rose = Plant("Rose", 25.0, 30)
rose.is_older_than_a_year(400)    # True
```

18. Les methodes de classe — @classmethod

```
# Recoit la CLASSE elle-meme comme premier parametre (cls)
# Sert souvent de constructeur alternatif

class Plant :
    @classmethod
    def create_anonymous(cls) -> 'Plant' :
        # cls = Plant (ou la classe enfant si heritee)
        return cls("Unknown", 0.0, 0)    # cree une instance avec valeurs par default

# Utilisation : creer sans avoir toutes les infos
mystery = Plant.create_anonymous()
mystery.show()    # Unknown: 0.0cm, 0 days old

# Avantage de cls vs Plant en dur :
# Si Flower herite et appelle Flower.create_anonymous(),
# cls vaudra Flower, pas Plant -> cree un Flower, pas un Plant !
```

19. Les classes imbriquees (nested classes)

```
# Une classe definie A L'INTERIEUR d'une autre classe
# Utile pour un sous-composant qui n'a de sens que rattache a la classe parente
```

```
class Plant :
    class Stats :                # indentee DANS Plant
        def __init__(self) -> None :
            self._grow_count = 0
            self._age_count = 0
            self._show_count = 0

        def grow(self) -> None :
            self._grow_count += 1

        def display(self, name: str) -> None :
            print(f"Stats for {name}:")
            print(f"grow() calls : {self._grow_count}")

    def __init__(self, name: str, height: float, age: int) -> None :
        self.name = name
        self._stats = Plant.Stats()    # chaque instance a SES PROPRES stats

    def display_stats(self) -> None :
        self._stats.display(self.name)    # methode publique pour acceder aux stats

    def grow(self) -> None :
        self._height = round(self._height + self.growth_rate, 1)
```

```

        self._stats.grow()      # incremente le compteur

# Heritage d'une classe imbriquee (ex: Tree.Tree_Stats herite de Plant.Stats)
class Tree(Plant) :
    class Tree_Stats(Plant.Stats) :      # herite de Plant.Stats
        def __init__(self) -> None :
            super().__init__()           # initialise les 3 compteurs parents
            self._produce_shade_count = 0   # compteur specifique a Tree

        def display(self, name: str) -> None :
            super().display(name)         # affiche les 3 premiers
            print(f"produce_shade() calls: {self._produce_shade_count}")

    def __init__(self, name: str, height: float, age: int, trunk_diameter: float) -> None :
        super().__init__(name, height, age)
        self._stats = Tree.Tree_Stats()  # REMPLACE Plant.Stats par Tree_Stats
        self.trunk_diameter = trunk_diameter

```

20. Les fonctions libres (hors classe)

```

# Definie au niveau racine, pas dans une classe
# Peut recevoir des objets en parametre et appeler leurs methodes

def display_plant_stats(plant: 'Plant') -> None :
    # Fonctionne avec Plant, Flower, Tree, Vegetable, Seed...
    # car ils heritent tous de Plant et ont tous display_stats()
    plant.display_stats()

# Utilisation
rose = Flower("Rose", 15.0, 10, "red")
display_plant_stats(rose)  # affiche les stats de rose

```

MODULE 02 — Gestion des exceptions

21. try / except — attraper une erreur

```
# Sans gestion d'erreurs : le programme crashe
temperature = int("abc") # CRASH ! Programme arrete

# Avec try/except : le programme continue
try :
    temperature = int("abc")
except ValueError as e :
    print(f"Erreur : {e}") # affiche le message d'erreur
# Le programme continue ici normalement

# Les types d'erreurs les plus courants :
# ValueError      : valeur du bon type mais impossible (int("abc"))
# TypeError       : types incompatibles ("a" + 1)
# ZeroDivisionError: division par zero (10 / 0)
# FileNotFoundError: fichier inexistant (open("inexistant.txt"))
# KeyError        : cle inexistant dans un dict (d["inexistant"])
# IndexError      : index hors limites (liste[100])
# AttributeError  : attribut/methode inexistant (42.upper())
# Exception       : classe de base de toutes les exceptions
```

22. raise — lancer une erreur volontairement

```
# raise : declencher une exception manuellement
# Utile pour valider des donnees et refuser une valeur invalide

def input_temperature(temp_str: str) -> int :
    temp = int(temp_str) # peut lever ValueError si "abc"
    if temp > 40 :
        raise ValueError(f"{temp}C is too hot for plants (max 40C)")
    if temp < 0 :
        raise ValueError(f"{temp}C is too cold for plants (min 0C)")
    return temp

try :
    t = input_temperature("100")
except ValueError as e :
    print(f"Caught: {e}")
# Caught: 100C is too hot for plants (max 40C)
```

23. Attraper plusieurs types d'erreurs

```
# Plusieurs blocs except separes
try :
    resultat = int(valeur) / diviseur
except ValueError :
    print("Valeur invalide")
except ZeroDivisionError :
```

```

        print("Division par zero")
except FileNotFoundError :
    print("Fichier introuvable")

# Plusieurs types dans un seul except (avec tuple)
try :
    resultat = int(valeur) / diviseur
except (ValueError, ZeroDivisionError) as e :
    print(f"Erreur : {e}")

```

24. finally — toujours nettoyer

```

# finally s'exécute TOUJOURS, erreur ou pas, return ou pas
# Parfait pour libérer des ressources

try :
    fichier = open("data.txt")
    donnees = fichier.read()
except FileNotFoundError :
    print("Fichier introuvable")
finally :
    print("Nettoyage")    # s'affiche TOUJOURS

# finally s'exécute même après un return dans except !
def exemple() -> str :
    try :
        raise ValueError("erreur")
    except ValueError :
        return "sortie"    # return préparé...
    finally :
        print("finally avant le return !")    # ...mais finally passe d'abord

# Ordre : try -> except -> finally -> return effectif

```

25. Les exceptions personnalisées

```

# Créer ses propres types d'exceptions
class GardenError(Exception) :
    def __init__(self, message: str = "Unknown garden error") -> None :
        super().__init__(message)

class PlantError(GardenError) :
    def __init__(self, message: str = "Unknown plant error") -> None :
        super().__init__(message)

class WaterError(GardenError) :
    def __init__(self, message: str = "Unknown water error") -> None :
        super().__init__(message)

# Utilisation
try :

```

```
        raise PlantError("La tomate est fanee !")
except PlantError as e :
    print(f"PlantError: {e}")

# Grace a l'heritage :
# attraper GardenError attrape aussi PlantError ET WaterError
try :
    raise WaterError("Pas assez d'eau !")
except GardenError as e :    # attrape WaterError car enfant de GardenError
    print(f"GardenError: {e}")

# Hierarchie :
# Exception -> GardenError -> PlantError
#                                     -> WaterError
```


MODULE 03 — Structures de donnees

26. sys.argv — arguments ligne de commande

```
import sys

# sys.argv : liste de tous les arguments passes au script
# python3 mon_script.py hello world 42

# sys.argv[0] = "mon_script.py" (toujours le nom du script)
# sys.argv[1] = "hello"
# sys.argv[2] = "world"
# sys.argv[3] = "42"
# len(sys.argv) = 4

# Parcourir les arguments (sans le nom du script)
for argument in sys.argv[1:] : # [1:] = du 2eme jusqu'a la fin
    print(argument)

# Verifier s'il y a des arguments
if len(sys.argv) == 1 :
    print("Aucun argument fourni")
else :
    print(f"Arguments: {len(sys.argv) - 1}")
```

27. Les listes

```
# Creer une liste
ma_liste = [] # liste vide
ma_liste = [1, 2, 3, "pomme"] # avec elements
scores = [85, 42, 67, 23]

# Acceder par index (commence a 0)
scores[0] # 85 (premier)
scores[-1] # 23 (dernier)
scores[1:3] # [42, 67] (slicing : du 2eme au 3eme)
scores[1:] # [42, 67, 23] (du 2eme jusqu'a la fin)

# Modifier
scores[0] = 100 # modifier un element
scores.append(50) # ajouter a la fin
scores.insert(0, 99) # inserer a l'index 0

# Infos utiles
len(scores) # nombre d'elements
sum(scores) # somme de tous les elements
max(scores) # valeur maximum
min(scores) # valeur minimum

# Parcourir
for score in scores :
```

```

    print(score)

# Convertir en liste
list(mon_dict.keys())    # cles d'un dict en liste
list(mon_set)            # set en liste

```

28. Les tuples

```

# Comme une liste mais IMMUTABLE (ne peut pas etre modifie)
# Parfait pour des donnees fixes comme des coordonnees (x,y,z)

# Creer un tuple
coordonnees = (10, 25, -5)
point = (1.5, 2.7, 3.1)

# Acceder par index (meme syntaxe que les listes)
coordonnees[0]    # 10 (x)
coordonnees[1]    # 25 (y)
coordonnees[2]    # -5 (z)

# Unpacking (deballage) : tres pratique !
x, y, z = coordonnees    # x=10, y=25, z=-5

# Longueur
len(coordonnees)    # 3

# On NE peut PAS modifier un tuple :
# coordonnees[0] = 99    # ERREUR : TypeError

# Retourner un tuple depuis une fonction
def get_position() -> tuple :
    return (1.0, 2.0, 3.0)

pos = get_position()    # pos = (1.0, 2.0, 3.0)

```

29. Les sets — ensembles

```

# Collection non ordonnee d'elements UNIQUES (pas de doublons)
# Equivalent d'un ensemble mathematique

# Creer un set
achievements = {"sword_master", "ninja", "explorer"}
vide = set()    # set vide (PAS {} qui cree un dict vide !)

# Ajouter un element
achievements.add("speed_runner")

# Les doublons sont automatiquement ignores
test = {"a", "b", "a", "c"}
print(test)    # {"a", "b", "c"}

# Convertir une liste en set (supprime les doublons)

```

```

ma_liste = [1, 2, 2, 3, 3]
mon_set = set(ma_liste)    # {1, 2, 3}

# Operations ensembles :
j1 = {"dragon", "explorer", "speed"}
j2 = {"dragon", "sword", "explorer"}
j3 = {"speed", "ninja", "dragon"}

# UNION : tous les elements de tous les sets (sans doublon)
set.union(j1, j2, j3)
# ou avec * pour deballer une collection de sets :
all_sets = [j1, j2, j3]
set.union(*all_sets)

# INTERSECTION : elements communs a TOUS les sets
set.intersection(j1, j2, j3)    # {"dragon"}

# DIFFERENCE : elements de j1 absents de j2 et j3
j1.difference(j2, j3)    # ce que j1 a exclusivement

# Trouver ce qui manque a j1 (par rapport a l'union totale)
all_achievements = set.union(j1, j2, j3)
all_achievements.difference(j1)    # ce que j1 n'a pas encore

```

30. Les dictionnaires

```

# Paires cle -> valeur
# Acces par cle (string, int...) pas par index numerique

# Creer
inventaire = {}                                # vide
inventaire = {"sword": 5, "potion": 3}    # avec valeurs initiales

# Acceder, ajouter, modifier
inventaire["sword"]            # 5
inventaire["shield"] = 2    # ajouter
inventaire["potion"] = 10    # modifier

# Verifier si une cle existe
"sword" in inventaire    # True
"axe" in inventaire      # False

# Parcourir
for cle in inventaire.keys() :    # parcourir les cles
    print(cle)

for valeur in inventaire.values() :    # parcourir les valeurs
    print(valeur)

for cle, valeur in inventaire.items() :    # parcourir les deux
    print(f"{cle}: {valeur}")

```

```
# Methodes utiles
list(inventaire.keys())    # ["sword", "potion", "shield"]
list(inventaire.values())  # [5, 10, 2]
sum(inventaire.values())   # 17 (somme des valeurs)

# Ajouter/modifier avec update()
inventaire.update({"bomb": 2})      # ajoute bomb:2
inventaire.update({"sword": 10})    # modifie sword

# Trouver la cle avec la valeur max (en gardant le premier en cas d'egalite)
item_max = list(inventaire.keys())[0]  # initialiser avec le premier
for nom, quantite in inventaire.items() :
    if quantite > inventaire[item_max] :
        item_max = nom    # remplacer seulement si STRICTEMENT superieur
                           # (> pas >= : garde le premier en cas d'egalite)
```

31. Erreurs classiques a eviter

```
# 1. Oublier de convertir le retour de input()
age = input("Age : ")
if age > 18 :    # ERREUR : comparaison str > int
    ...
# FIX :
if int(age) > 18 :    # OK

# 2. Conflit nom methode / attribut dans une classe
class Plant :
    def __init__(self) :
        self.age = 30    # attribut "age"
    def age(self) :    # methode "age" : ECRASE l'attribut !
        self._age += 1
# FIX : nommer differemment -> def age_grow(self)

# 3. Oublier super().__init__() dans une classe enfant
class Flower(Plant) :
    def __init__(self, color) :
        # OUBLI : self.name, self._height... n'existent pas !
        self.color = color
# FIX :
    def __init__(self, name, height, age, color) :
        super().__init__(name, height, age)    # toujours en premier !
        self.color = color

# 4. Imprecisions des flottants
0.1 + 0.2    # 0.30000000000000004 (pas 0.3 !)
# FIX : utiliser round()
round(0.1 + 0.2, 1)    # 0.3

# 5. Mauvaise indentation (code dans la boucle au lieu d'apres)
for argument in sys.argv[1:] :
    inventory[item] = quantity
    print(inventory)    # MAUVAIS : affiche a chaque iteration !
# FIX :
for argument in sys.argv[1:] :
    inventory[item] = quantity
print(inventory)    # BON : affiche apres la boucle

# 6. Attraper Exception sans raison
try :
    ...
except Exception :
    pass    # TRES MAUVAIS : masque tous les bugs !
# FIX : attraper le type precis
except ValueError as e :
    print(f"Erreur : {e}")

# 7. set() vide vs dict vide
```

```

vide_set = set()    # set vide : CORRECT
vide_set = {}       # FAUX : cree un dict vide, pas un set !
vide_dict = {}      # dict vide : CORRECT

```

32. Bonnes pratiques PEP 8

```

# NOMMAGE
ma_variable = 42          # snake_case pour variables et fonctions
ma_fonction()             # snake_case
class MaClasse :          # PascalCase pour les classes
MA_CONSTANTE = 3.14       # SCREAMING_SNAKE_CASE pour les constantes

# A EVITER (autres langages) :
maVariable = 42           # camelCase (JavaScript)
isOlderThanAYear()        # camelCase (non en Python)

# TYPE HINTS (annotations de type)
def ma_fonction(nom: str, age: int) -> None :
    ...
def get_height(self) -> float :
    return self._height
def input_temperature(temp_str: str) -> int :
    return int(temp_str)

# STRUCTURE D'UN FICHIER PYTHON
#!/usr/bin/env python3    # 1. shebang
import sys                # 2. imports
import math

class Plant :              # 3. classes
    ...
class Flower(Plant) :
    ...

def fonction_libre() :     # 4. fonctions libres
    ...

if __name__ == "__main__" : # 5. point d'entree (toujours en dernier)
    main()

# OPERATEUR * pour deballer une collection
sets = [set1, set2, set3]
set.union(*sets)          # equivalent de set.union(set1, set2, set3)

# SLICING (decoupage de liste)
sys.argv[1:]              # tous les elements sauf le premier (le nom du script)
ma_liste[2:5]              # elements de l'index 2 a 4 inclus
ma_liste[:3]               # les 3 premiers elements
ma_liste[-1]              # le dernier element

```

Bonne chance pour la suite et l'entretien Odoo Belgium ! Courage ! ■